

Chat API Integration Guide

This guide is for the mobile developer implementing the new task chat feature in both the Client app and the Provider app.

Summary

- Chat is task-scoped.
- A chat room is created automatically when an offer is accepted and a provider is assigned to a task.
- Chat is REST-only. There is no WebSocket or SignalR real-time stream.
- Text messages are stored in the backend.
- Price-change and reschedule request cards are virtual thread items projected from their existing request tables.
- The chat history endpoint returns a mixed thread of text messages and request cards.
- Mobile apps must use `itemType` to decide how to render each thread item.
- Firebase notifications are sent for new text messages to the other chat participant only.
- Firebase notifications for price-change and reschedule cards are handled by the existing request notification flows.

Authentication

All chat endpoints require the normal bearer token for the logged-in user.

Client app:

```
Authorization: Bearer <client-token>
```

Provider app:

```
Authorization: Bearer <provider-token>
```

Role requirements:

- Client endpoints require role `Client`.
- Provider endpoints require role `Provider`.

All chat data is scoped to the authenticated user. A user can only list, open, and send messages in conversations where they are a participant.

Base Endpoints

Client App

- GET `/api/v1.0/client/chats`
- GET `/api/v1.0/client/chats/{conversationId}`
- GET `/api/v1.0/client/chats/tasks/{taskId}`
- GET `/api/v1.0/client/chats/{conversationId}/messages`
- POST `/api/v1.0/client/chats/{conversationId}/messages`

Provider App

- GET `/api/v1.0/provider/chats`
- GET `/api/v1.0/provider/chats/{conversationId}`
- GET `/api/v1.0/provider/chats/tasks/{taskId}`
- GET `/api/v1.0/provider/chats/{conversationId}/messages`
- POST `/api/v1.0/provider/chats/{conversationId}/messages`

Pagination

List endpoints support query parameters:

- `pageNumber` optional, default `1`
- `pageSize` optional, default `25`

Example:

```
GET /api/v1.0/client/chats?pageNumber=1&pageSize=25
Authorization: Bearer <client-token>
```

Paginated responses have this shape:

```
{
  "items": [],
  "totalCount": 0,
  "pageNumber": 1,
  "pageSize": 25,
  "totalPages": 0,
  "hasPreviousPage": false,
  "hasNextPage": false
}
```

Conversation List

Use this endpoint to show the user's conversation inbox.

Client

```
GET /api/v1.0/client/chats?pageNumber=1&pageSize=25
Authorization: Bearer <client-token>
```

Provider

```
GET /api/v1.0/provider/chats?pageNumber=1&pageSize=25
Authorization: Bearer <provider-token>
```

Success response:

- 200 OK

Example:

```
{
  "items": [
    {
      "id": "aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa",
      "taskID": "11111111-1111-1111-1111-111111111111",
      "taskTitle": "Fix kitchen sink",
      "taskStatus": "InProgress",
      "clientID": "22222222-2222-2222-2222-222222222222",
      "clientName": "Sara Mohamed",
      "providerID": "33333333-3333-3333-3333-333333333333",
      "providerName": "Ahmed Ali",
      "lastItem": {
        "id": "bbbbbbbbb-bbbb-bbbb-bbbb-bbbbbbbbbbbbb",
        "conversationID": "aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa",
        "taskID": "11111111-1111-1111-1111-111111111111",
        "itemType": "text",
        "createdAt": "2026-05-02T13:20:00Z",
        "message": {
          "id": "bbbbbbbbb-bbbb-bbbb-bbbb-bbbbbbbbbbbbb",
          "conversationID": "aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa",
          "senderID": "33333333-3333-3333-3333-333333333333",
          "senderName": "Ahmed Ali",
          "content": "I am on my way.",
          "contentType": "text",
          "timestamp": "2026-05-02T13:20:00Z"
        },
        "card": null
      },
      "createdAt": "2026-05-02T12:00:00Z"
    }
  ],
  "totalCount": 1,
  "pageNumber": 1,
  "pageSize": 25,
  "totalPages": 1,
  "hasPreviousPage": false,
  "hasNextPage": false
}
```

Conversation Fields

- `id`: conversation ID. Use this as `conversationId` in chat endpoints.
- `taskID`: task ID linked to this chat.
- `taskTitle`: task title for display.
- `taskStatus`: current task status.
- `clientID`: client user ID.
- `clientName`: client display name.
- `providerID`: hired provider user ID.
- `providerName`: provider display name.
- `lastItem`: latest renderable thread item. Can be a text message or a card.
- `createdAt`: conversation creation timestamp in UTC.

Get Conversation By ID

Use this when opening a known conversation.

Client

```
GET /api/v1.0/client/chats/{conversationId}
Authorization: Bearer <client-token>
```

Provider

```
GET /api/v1.0/provider/chats/{conversationId}
Authorization: Bearer <provider-token>
```

Success response:

- 200 OK
- Body: one ChatConversationDto

Common errors:

- 401 token missing or invalid
- 403 token has the wrong role
- 404 conversation does not exist or does not belong to the authenticated user

Get Conversation By Task ID

Use this when the app is on a task details screen and needs to open the task chat.

Client

```
GET /api/v1.0/client/chats/tasks/{taskId}
Authorization: Bearer <client-token>
```

Provider

```
GET /api/v1.0/provider/chats/tasks/{taskId}
Authorization: Bearer <provider-token>
```

Success response:

- 200 OK
- Body: one ChatConversationDto

Important:

- This endpoint does not create missing chat rooms.
- If the task was assigned before chat was introduced and has no conversation, expect 404 .
- If the authenticated user is not the task client or hired provider, expect 404 .

Chat Thread / History

Use this endpoint to render the chat screen.

The route is named /messages , but the response is a mixed thread containing:

- text message items
- price-change request card items
- reschedule request card items

Client

```
GET /api/v1.0/client/chats/{conversationId}/messages?pageNumber=1&pageSize=25
Authorization: Bearer <client-token>
```

Provider

```
GET /api/v1.0/provider/chats/{conversationId}/messages?pageNumber=1&pageSize=25
Authorization: Bearer <provider-token>
```

Success response:

- 200 OK
- Body: PaginatedResponse<ChatThreadItemDto>

Items are ordered oldest first:

1. createdAt ascending
2. itemType
3. id

Example response:

```
{
  "items": [
    {
      "id": "bbbbbbbb-bbbb-bbbb-bbbb-bbbbbbbbbbbb",
      "conversationID": "aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa",
      "taskID": "11111111-1111-1111-1111-111111111111",
      "itemType": "text",
      "createdAt": "2026-05-02T13:20:00Z",
      "message": {
        "id": "bbbbbbbb-bbbb-bbbb-bbbb-bbbbbbbbbbbb",
        "conversationID": "aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa",
        "senderID": "33333333-3333-3333-3333-333333333333",
        "senderName": "Ahmed Ali",
        "content": "I am on my way.",
        "contentType": "text",
        "timestamp": "2026-05-02T13:20:00Z"
      },
      "card": null
    },
    {
      "id": "cccccccc-cccc-cccc-cccc-cccccccccccc",
      "conversationID": "aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa",
      "taskID": "11111111-1111-1111-1111-111111111111",
      "itemType": "price_change_request_card",
      "createdAt": "2026-05-02T13:30:00Z",
      "message": null,
      "card": {
        "cardType": "price_change_request",
        "status": "Pending",
        "requestedById": "33333333-3333-3333-3333-333333333333",
        "requestedByName": "Ahmed Ali",
        "isInitiatedByClient": false,
        "reason": "Extra replacement parts are needed.",
        "canAccept": true,
        "canReject": true,
        "canCounter": true,
        "priceChange": {
          "priceChangeRequestId": "cccccccc-cccc-cccc-cccc-cccccccccccc",
          "originalAmount": 150.0,
          "requestedAmount": 190.0,
          "currency": "LYD"
        }
      },
      "reschedule": null
    }
  ],
  {
    "id": "dddddddd-dddd-dddd-dddd-dddddddddddd",
    "conversationID": "aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa",
    "taskID": "11111111-1111-1111-1111-111111111111",
    "itemType": "reschedule_request_card",
    "createdAt": "2026-05-02T13:45:00Z",
    "message": null,
    "card": {
      "cardType": "reschedule_request",
      "status": "Pending",
      "requestedById": "22222222-2222-2222-2222-222222222222",
      "requestedByName": "Sara Mohamed",
      "isInitiatedByClient": true,
      "reason": "I will not be home at the original time.",
      "canAccept": true,
      "canReject": true,
      "canCounter": true,
      "priceChange": null,
      "reschedule": {
```

```

      reschedule : {
        "rescheduleRequestId": "ddddddd-dddd-dddd-dddddddddd",
        "originalScheduledDateTime": "2026-05-03T10:00:00Z",
        "requestedScheduledDateTime": "2026-05-03T14:00:00Z"
      }
    }
  ],
  "totalCount": 3,
  "pageNumber": 1,
  "pageSize": 25,
  "totalPages": 1,
  "hasPreviousPage": false,
  "hasNextPage": false
}

```

Thread Item Discriminator

Every thread item has `itemType`.

Supported values:

- `text`
- `price_change_request_card`
- `reschedule_request_card`

Rendering rule:

- If `itemType` = `"text"`, render `message`.
- If `itemType` = `"price_change_request_card"`, render `card.priceChange`.
- If `itemType` = `"reschedule_request_card"`, render `card.reschedule`.

The app should not infer card type from nullable fields. Use `itemType` and `card.cardType`.

Text Message Item

Text item shape:

```

{
  "id": "<message-guid>",
  "conversationID": "<conversation-guid>",
  "taskID": "<task-guid>",
  "itemType": "text",
  "createdAt": "2026-05-02T13:20:00Z",
  "message": {
    "id": "<message-guid>",
    "conversationID": "<conversation-guid>",
    "senderID": "<user-guid>",
    "senderName": "Ahmed Ali",
    "content": "Text message",
    "contentType": "text",
    "timestamp": "2026-05-02T13:20:00Z"
  },
  "card": null
}

```

Fields:

- `message.contentType` is currently always `text` for user-sent chat messages.
- Use `senderID` to decide whether to render the message as mine/theirs.
- Use `timestamp` or parent `createdAt` for display. They represent the same event time for text messages.

Price Change Request Card

Price-change card item shape:

```
{
  "id": "<price-change-request-guid>",
  "conversationID": "<conversation-guid>",
  "taskId": "<task-guid>",
  "itemType": "price_change_request_card",
  "createdAt": "2026-05-02T13:30:00Z",
  "message": null,
  "card": {
    "cardType": "price_change_request",
    "status": "Pending",
    "requestedById": "<user-guid>",
    "requestedByName": "Ahmed Ali",
    "isInitiatedByClient": false,
    "reason": "Extra replacement parts are needed.",
    "canAccept": true,
    "canReject": true,
    "canCounter": true,
    "priceChange": {
      "priceChangeRequestId": "<price-change-request-guid>",
      "originalAmount": 150.0,
      "requestedAmount": 190.0,
      "currency": "LYD"
    },
    "reschedule": null
  }
}
```

Statuses:

- Pending
- Accepted
- Rejected

Display suggestions:

- Title: "Price change request"
- Show original amount and requested amount.
- Show reason if present.
- Show status badge.
- Show buttons based only on canAccept, canReject, and canCounter.

Reschedule Request Card

Reschedule card item shape:

```
{
  "id": "<reschedule-request-guid>",
  "conversationID": "<conversation-guid>",
  "taskId": "<task-guid>",
  "itemType": "reschedule_request_card",
  "createdAt": "2026-05-02T13:45:00Z",
  "message": null,
  "card": {
    "cardType": "reschedule_request",
    "status": "Pending",
    "requestedById": "<user-guid>",
    "requestedByName": "Sara Mohamed",
    "isInitiatedByClient": true,
    "reason": "I will not be home at the original time.",
    "canAccept": true,
    "canReject": true,
    "canCounter": true,
    "priceChange": null,
    "reschedule": {
      "rescheduleRequestId": "<reschedule-request-guid>",
      "originalScheduledDateTime": "2026-05-03T10:00:00Z",
      "requestedScheduledDateTime": "2026-05-03T14:00:00Z"
    }
  }
}
```

Statuses:

- Pending
- Accepted
- Rejected

Display suggestions:

- Title: "Reschedule request"
- Show original scheduled time and requested scheduled time.
- Show `reason` if present.
- Show status badge.
- Show buttons based only on `canAccept`, `canReject`, and `canCounter`.

Card Button Rules

The backend computes the action flags for the authenticated user.

Use these flags directly:

- Show Accept if `card.canAccept = true`.
- Show Reject if `card.canReject = true`.
- Show Counter if `card.canCounter = true`.

Do not show action buttons when:

- `status` is not Pending.
- The current user created the request.
- The corresponding `can*` flag is `false`.

Counter behavior:

- There is no backend "counter request" endpoint in this release.
- The Counter button should open the app's existing create request form.
- After creating a counter request, refresh the chat thread.

Price Change Card Actions

Use `card.priceChange.priceChangeRequestId` and the parent `taskId`.

Client App

Accept:

```
POST /api/v1.0/client/tasks/{taskId}/price-change-requests/{requestId}/accept
Authorization: Bearer <client-token>
Content-Type: application/json
```

Reject:

```
POST /api/v1.0/client/tasks/{taskId}/price-change-requests/{requestId}/reject
Authorization: Bearer <client-token>
Content-Type: application/json
```

Optional request body for accept/reject:

```
{
  "responseNote": "Optional note"
}
```

Create new price-change request, including counter flow:

```
POST /api/v1.0/client/tasks/{taskId}/price-change-requests
Authorization: Bearer <client-token>
Content-Type: application/json
```

```
{
  "requestedAmount": 190.0,
  "reason": "Optional reason"
}
```

Provider App

Accept:

```
POST /api/v1.0/provider/tasks/{taskId}/price-change-requests/{requestId}/accept
Authorization: Bearer <provider-token>
Content-Type: application/json
```

Reject:

```
POST /api/v1.0/provider/tasks/{taskId}/price-change-requests/{requestId}/reject
Authorization: Bearer <provider-token>
Content-Type: application/json
```

Optional request body for accept/reject:

```
{
  "responseNote": "Optional note"
}
```

Create new price-change request, including counter flow:

```
POST /api/v1.0/provider/tasks/{taskId}/price-change-requests
Authorization: Bearer <provider-token>
Content-Type: application/json
```

```
{
  "requestedAmount": 190.0,
  "reason": "Optional reason"
}
```

Reschedule Card Actions

Use `card.reschedule.rescheduleRequestId` and the parent `taskId`.

Client App

Accept:

```
POST /api/v1.0/client/tasks/{taskId}/reschedule-requests/{requestId}/accept
Authorization: Bearer <client-token>
Content-Type: application/json
```

Reject:

```
POST /api/v1.0/client/tasks/{taskId}/reschedule-requests/{requestId}/reject
Authorization: Bearer <client-token>
Content-Type: application/json
```

Optional request body for accept/reject:

```
{
  "responseNote": "Optional note"
}
```

Create new reschedule request, including counter flow:

```
POST /api/v1.0/client/tasks/{taskId}/reschedule-requests
Authorization: Bearer <client-token>
Content-Type: application/json
```

```
{
  "requestedScheduledDateTime": "2026-05-03T14:00:00Z",
  "reason": "Optional reason"
}
```

Provider App

Accept:

```
POST /api/v1.0/provider/tasks/{taskId}/reschedule-requests/{requestId}/accept
Authorization: Bearer <provider-token>
Content-Type: application/json
```

Reject:

```
POST /api/v1.0/provider/tasks/{taskId}/reschedule-requests/{requestId}/reject
Authorization: Bearer <provider-token>
Content-Type: application/json
```

Optional request body for accept/reject:

```
{
  "responseNote": "Optional note"
}
```

Create new reschedule request, including counter flow:

```
POST /api/v1.0/provider/tasks/{taskId}/reschedule-requests
Authorization: Bearer <provider-token>
Content-Type: application/json
```

```
{
  "requestedScheduledDateTime": "2026-05-03T14:00:00Z",
  "reason": "Optional reason"
}
```

Send Text Message

Use this endpoint to send a normal text message.

Client

```
POST /api/v1.0/client/chats/{conversationId}/messages
Authorization: Bearer <client-token>
Content-Type: application/json
```

Provider

```
POST /api/v1.0/provider/chats/{conversationId}/messages
Authorization: Bearer <provider-token>
Content-Type: application/json
```

Request body:

```
{
  "content": "Hello, I am on my way."
}
```

Validation:

- `content` is required.
- `content` cannot be whitespace only.
- Maximum length is 4000 characters.

Success response:

- 201 Created
- Body: ChatMessageDto

Example:

```
{
  "id": "bbbbbbbbb-bbbb-bbbb-bbbb-bbbbbbbbbbbb",
  "conversationID": "aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa",
  "senderID": "33333333-3333-3333-3333-333333333333",
  "senderName": "Ahmed Ali",
  "content": "Hello, I am on my way.",
  "contentType": "text",
  "timestamp": "2026-05-02T13:20:00Z"
}
```

Important:

- This endpoint only sends text messages.
- It does not create price-change or reschedule cards.
- To create a card, use the price-change or reschedule request create endpoint.

Firestore Notifications

Text Chat Message Notification

When a user sends a text chat message, the backend queues a Firestore notification to the other conversation participant only.

The sender does not receive their own push notification.

Notification type:

- `chat_message`

FCM data payload:

```
{
  "type": "chat_message",
  "conversationId": "<conversation-guid>",
  "messageId": "<message-guid>",
  "taskId": "<task-guid>",
  "senderId": "<user-guid>"
}
```

Recommended app behavior:

1. If already on the same conversation screen, refresh or append after fetching the latest thread page.
2. If not on the chat screen, navigate to the task chat using `conversationId`.
3. Fetch `GET /api/v1.0/{role}/chats/{conversationId}/messages` to get the authoritative thread.

Price Change And Reschedule Notifications

Request cards do not produce separate `chat_message` notifications.

They use existing notification types:

- `task_price_change_request_created`
- `task_price_change_request_accepted`
- `task_price_change_request_rejected`
- `task_reschedule_request_created`
- `task_reschedule_request_accepted`
- `task_reschedule_request_rejected`

Recommended app behavior:

- Route to task details or chat thread.
- Refresh the chat thread to show the request card and current live status.

Recommended Mobile Flow

Open Chat From Task Details

1. Call `GET /api/v1.0/{role}/chats/tasks/{taskId}`.
2. If `200`, use returned id as `conversationId`.
3. Call `GET /api/v1.0/{role}/chats/{conversationId}/messages`.
4. Render each item using `itemType`.
5. If `404`, hide or disable chat for that task.

Open Chat From Inbox

1. Call `GET /api/v1.0/{role}/chats`.
2. Render each conversation using `taskTitle`, `counterparty name`, and `lastItem`.
3. When a user opens a row, call `GET /api/v1.0/{role}/chats/{conversationId}/messages`.

After Sending Text

1. Optimistically add the text locally if desired.
2. Call `POST /api/v1.0/{role}/chats/{conversationId}/messages`.
3. Replace the optimistic item with the backend response.
4. Refresh the thread if ordering or cards may have changed.

After Accepting Or Rejecting A Card

1. Call the relevant accept/reject endpoint.
2. Refresh the chat thread.
3. The same card should now show updated `status`.
4. Hide action buttons because `canAccept`, `canReject`, and `canCounter` should become `false`.

After Creating A Price Change Or Reschedule Request

1. Call the relevant create endpoint.
2. Refresh the chat thread.
3. A new virtual card should appear in chronological order.

Error Behavior

Common chat endpoint errors:

- `400` invalid message body or conversation is not messageable

- 401 token missing or invalid
- 403 token has the wrong role or user is not allowed
- 404 conversation not found, task chat not found, or user is not a participant
- 500 unexpected server error

Common request card action errors:

- 400 task is not active
- 400 requested amount/date is invalid
- 400 request cannot be created for the task state
- 403 user is trying to respond to their own request
- 404 task or request not found
- 409 there is already a pending request or the request is no longer pending

Recommended behavior:

- On 404 for chat by task, hide chat entry point for that task.
- On 409 for card actions, refresh the chat thread because request state likely changed.
- On 403 for card actions, refresh the chat thread and hide actions.

Suggested Mobile Models

TypeScript-style example:

```
type PaginatedResponse<T> = {
  items: T[];
  totalCount: number;
  pageNumber: number;
  pageSize: number;
  totalPages: number;
  hasPreviousPage: boolean;
  hasNextPage: boolean;
};

type ChatConversation = {
  id: string;
  taskID: string;
  taskTitle: string | null;
  taskStatus: string | null;
  clientID: string;
  clientName: string;
  providerID: string;
  providerName: string;
  lastItem: ChatThreadItem | null;
  createdAt: string;
};

type ChatThreadItem =
  | {
    id: string;
    conversationID: string;
    taskID: string;
    itemType: "text";
    createdAt: string;
    message: ChatMessage;
    card: null;
  }
  | {
    id: string;
    conversationID: string;
    taskID: string;
    itemType: "price_change_request_card" | "reschedule_request_card";
    createdAt: string;
    message: null;
    card: ChatCard;
  };

type ChatMessage = {
  id: string;
```

```

conversationID: string;
senderID: string;
senderName: string;
content: string;
contentType: "text";
timestamp: string;
};

type ChatCard = {
  cardType: "price_change_request" | "reschedule_request";
  status: "Pending" | "Accepted" | "Rejected";
  requestedById: string;
  requestedByName: string;
  isInitiatedByClient: boolean;
  reason: string | null;
  canAccept: boolean;
  canReject: boolean;
  canCounter: boolean;
  priceChange: PriceChangeCard | null;
  reschedule: RescheduleCard | null;
};

type PriceChangeCard = {
  priceChangeRequestId: string;
  originalAmount: number;
  requestedAmount: number;
  currency: string;
};

type RescheduleCard = {
  rescheduleRequestId: string;
  originalScheduledDateTime: string;
  requestedScheduledDateTime: string;
};

```

Implementation Checklist

Client App

- Add chat inbox using GET /api/v1.0/client/chats .
- Add task chat lookup using GET /api/v1.0/client/chats/tasks/{taskId} .
- Add chat thread screen using GET /api/v1.0/client/chats/{conversationId}/messages .
- Render text, price_change_request_card, and reschedule_request_card .
- Wire card accept/reject buttons to client task request endpoints.
- Wire card counter button to open the relevant create request form.
- Handle chat_message FCM payload and refresh the thread.

Provider App

- Add chat inbox using GET /api/v1.0/provider/chats .
- Add task chat lookup using GET /api/v1.0/provider/chats/tasks/{taskId} .
- Add chat thread screen using GET /api/v1.0/provider/chats/{conversationId}/messages .
- Render text, price_change_request_card, and reschedule_request_card .
- Wire card accept/reject buttons to provider task request endpoints.
- Wire card counter button to open the relevant create request form.
- Handle chat_message FCM payload and refresh the thread.

Important Non-Features In This Release

- No read receipts.
- No unread counts.
- No typing indicator.
- No attachments.
- No message edit/delete.
- No WebSocket or SignalR real-time channel.
- No server-side counter endpoint for request cards.
- No backfill for old assigned tasks without chat rooms.